Technology Stack & Rationale

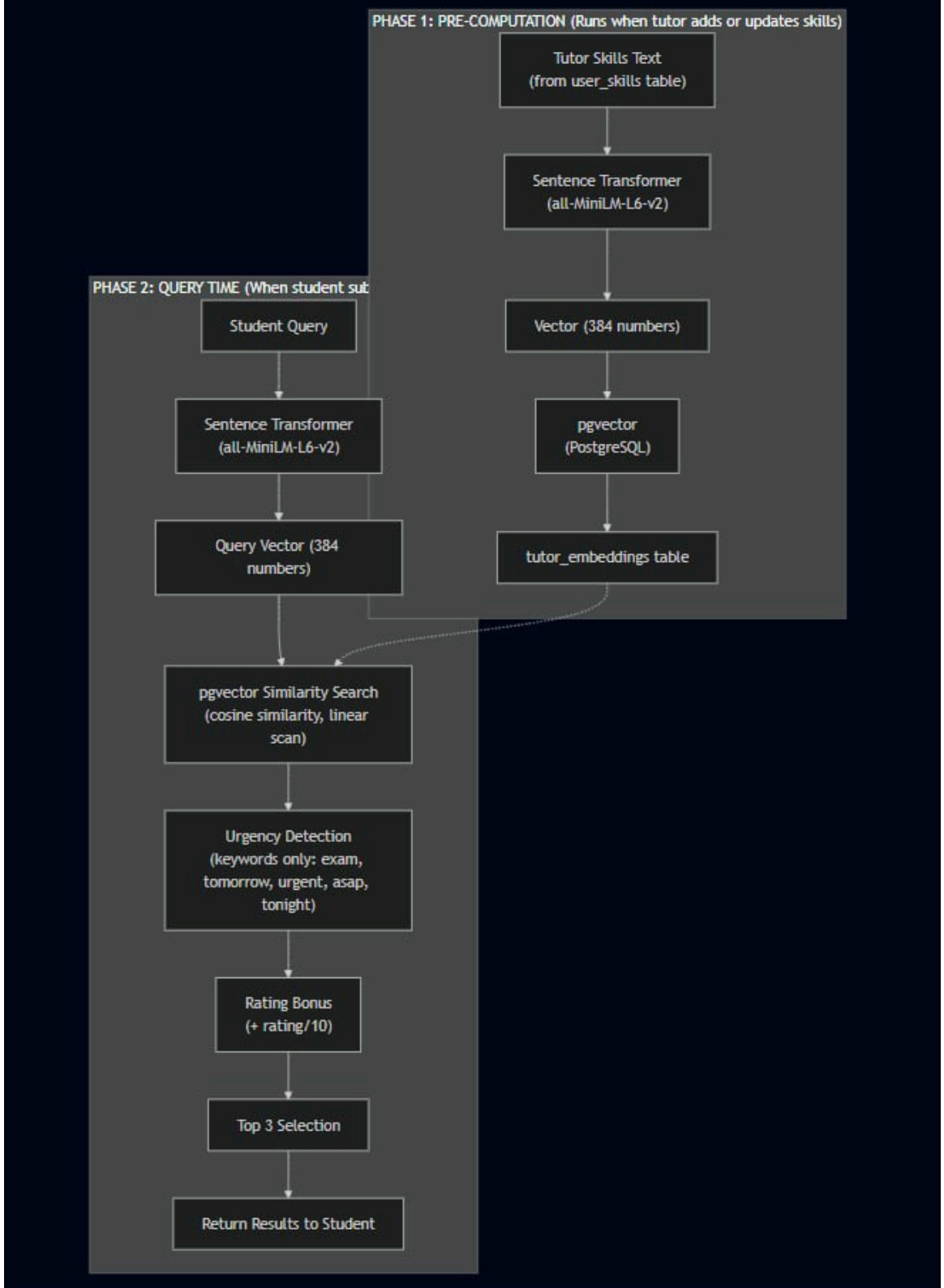
Layer	Technology	Rationale
Frontend	React	Provides a smooth, app-like experience without page reloads. Component-based architecture matches the modular backend.
Backend	Python + Django	Django includes built-in admin, authentication, and ORM – reducing the amount of code required.
API Framework	Django REST Framework (DRF)	Built on top of Django. Provides serializers for JSON validation and token authentication. The browsable API helps with testing.
Database	PostgreSQL+pgvector	Relational model matches our data. Supports UUID v7 (NFR-04). Handles 50 concurrent users (NFR-08). Free. pgvector extension enables vector storage for AI embeddings (384 dimensions) – allows similarity search directly in PostgreSQL without adding a separate vector database..
Authentication	JWT + Django Sessions	JWT provides stateless authentication for the frontend. Django sessions support the admin panel for verifying tutor qualifications and managing reported content. Using both is standard when combining Django REST Framework with the built-in admin.
AI / NLP	Semantic search with embeddings (Sentence Transformers (SBERT) with all-MiniLM-L6-v2)	Converts student queries and tutor skills to embeddings (vectors) for semantic search. Understands meaning, not just keywords. Pre-trained model runs locally. Adds urgency detection via keyword matching. Returns top 3 tutors based on similarity + rating + urgency.
File Storage	Django FileField + local storage	Built into Django. No external services to configure. Qualification files are small (PDF, JPG, PNG under 5MB).
Email	Gmail SMTP	Django has built-in email support. Free tier (100–500 emails/day) is sufficient for now. Configuration takes minutes.
Version Control	Git + GitHub	Standard tool. Free for students. Already used since project start.

What We're NOT Using (and Why)

Technology	Why Rejected
Other databases (MongoDB, MySQL, SQLite, NoSQL)	Our data has clear relationships (users → bookings → reviews). PostgreSQL supports foreign keys, joins, and UUID v7 (required by NFR-04) natively. MongoDB would require embedding or multiple queries. MySQL has less support for JSON and UUID. SQLite does not handle concurrent writes well (NFR-08). NoSQL databases would force data duplication or multiple queries, increasing complexity and risking inconsistency. PostgreSQL meets all our requirements.
React (if Bootstrap chosen)	Bootstrap with HTML/CSS is sufficient for our responsive design needs (NFR-06). Adding React would introduce a framework learning curve and increase frontend complexity without a clear benefit for our feature set. Page reloads are acceptable for a student project of this scale.
Cloud storage (S3, Cloudinary)	Adds cost and complexity. Local file storage using Django's FileField is enough for our qualification uploads. We do not need CDN delivery or advanced image processing.
Node.js / Express	Our team is learning Python for backend development. Adding another language would split our learning effort and increase coordination complexity. Django also provides built-in features (admin, auth, ORM) that Node.js lacks.
Background jobs (Celery, Redis, RabbitMQ, etc.)	Email sending completes within seconds. AI will run within NFR-02 (4 seconds). No long-running tasks. Avoiding background jobs keeps the system simple and easier to debug.
TextBlob / spaCy for AI	These are keyword-based NLP libraries. They do NOT perform semantic search or embedding-based retrieval. TextBlob extracts noun phrases but does not understand meaning.
OpenAI Embeddings API	While it would work, it requires an internet connection and API key, and has cost. Sentence Transformers runs locally, is free, and meets the same requirement without external dependencies
Separate Vector Database (Pinecone, Qdrant, Milvus)	PostgreSQL with pgvector handles our vector storage needs (<100 tutors, <1000 vectors). Adding a separate vector database would increase deployment complexity, require learning a new system, and is unnecessary for our scale. We would consider this only if we had millions of vectors or needed sub-millisecond latency.

AI Recommendations pipleline:

The AI pipeline has two phases. In Phase 1 (pre-computation), tutor skills are converted to 384-dimension vectors using Sentence Transformer and stored in PostgreSQL with pgvector. This happens only when tutors add or update their skills. In Phase 2 (query time), the student's query is converted to a vector and compared against all stored tutor vectors using cosine similarity. Results are then boosted by urgency (if keywords like 'exam' are detected, available tutors get +0.3) and rating (rating ÷ 10). The top 3 tutors are returned to the student.


PHASE 1: PRE-COMPUTATION (Runs when tutor adds or updates skills)
Tutor Skills Text (from user_skills table)
Sentence Transformer (all-MiniLM-L6-v2)
Vector (384 numbers)
pgvector (PostgreSQL)
tutor_embeddings table
PHASE 2: QUERY TIME (When student sut
Student Query
Sentence Transformer (all-MiniLM-L6-v2)
Query Vector (384 numbers)
pgvector Similarity Search (cosine similarity, linear scan)
Urgency Detection (keywords only: exam, tomorrow, urgent, asap, tonight)
Rating Bonus (+ rating/10)
Top 3 Selection
Return Results to Student

Explanation of urgency+Rating boost:

Boost	When Applied	How It Affects Ranking
Urgency Bonus	If student query contains keywords like "exam", "tomorrow", "urgent", "asap", "tonight"	Tutors available now receive a score boost (+0.3), moving them higher in results
Rating Bonus	Always applied (all tutors have ratings)	Tutors with higher average ratings receive a proportional boost (rating ÷ 10), encouraging quality tutors to appear first